Generating Hypercubes of Type Systems

Michael Stay
director.research@f1r3fly.io

L. Gregory Meredith
f1r3flyceo@gmail.com

Christian Wells
cb@holdea.net

Abstract

Barendregt's Lambda Cube organizes typed calculi by enabling different dependencies between terms and types. This paper shows how to derive an analogous *hypercube* of type systems for an arbitrary operational theory.

1 Introduction

Since the late 1980s, Barendregt's Lambda Cube[4] has served as a compact "map" of typed λ-calculi: by toggling three fundamental dependencies—types on terms, terms on types, and types on types—one obtains eight canonical systems. The Cube's lasting value is not only in its ability to classify; it provides a geometric way of thinking about design choices that later became systematized by the framework of Pure Type Systems (PTS), where a single parametric scheme generates many calculi from choices of sorts, axioms, and typing rules.

This paper asks for an analogous organizing principle in settings where the primary presentation is *operational*: a language is given by term formers, equations, and one-step rewrite rules, sometimes far from the λ-calculus in behavior (e.g. process calculi or hybrids of multiple models of computation like functional reactive programming). In such cases we would like a principled way to *extract* a family of type systems from the operational rules themselves, rather than imposing a bespoke typing discipline from the outside.

Our starting point is a lambda theory T: a finite-limits category with a chosen cartesian closed structure, a distinguished object Pr of processes, and an internal one-step reduction relation $\rightsquigarrow$. We reason in the internal language using judgments of the form

$$\Gamma \mid \Delta \vdash \psi,$$

where Γ is a variable context, Δ is a "rely" context of assumed propositions (including reductions and typings), and ψ is the conclusion. Base rewrites have the schematic form

$$x_1 : X_1, \ldots, x_n : X_n \mid \top \vdash L(\vec{x}) \rightsquigarrow R(\vec{x}),$$

and, crucially, each subterm occurrence on the left-hand side determines a one-hole context $C[-]$ in which that redex position sits.

From this operational data we define a free extension that generates a *modal layer* of type formers[1, 3]. For each base rewrite and each chosen redex position, we introduce a primitive modality

$$\langle K \rangle_{\vec{x} :: \vec{A}} B,$$

read as a *rely-possibly* specification: under rely assumptions $\vec{x} :: \vec{A}$, placing a term into the context $K[-]$ takes one step to a specified right-hand side inhabiting B. Rewrite-generated modalities do *not* support full conversion; instead, they support a weakened conversion principle via an empty-context possibility operator $\Diamond$, with $t :: \Diamond B$ expressing that t has a one-step reduct inhabiting B. Equation-generated modalities *do* support full conversion by the usual conversion rule.

Why does this produce a *hypercube*? Each generated type former carries finitely many $*/\Box$ *sort slots*: one slot for each rely input and one output slot. Raw slot-fillings form a Boolean cube of choices, but operational and equational laws force coherence: only those global slot assignments that preserve sort-level interpretations of equations and rewrites are admissible. We package these admissible assignments as an *equational center* $\mathcal{Z}$, and the vertices of the resulting (hyper)cube are the elements of $\mathcal{Z}$. In a cartesian closed setting, the canonical equation modality induced by evaluation yields a true dependent product Π; the induced cube recovers Barendregt's familiar organization. More generally, the cube's axes are not postulated: they are determined locally by the free variables of operational contexts and the result types demanded by the rules.

Two orthogonal extensions increase expressivity in a controlled way while preserving the same slot-and-center geometry. First, we add *structural* (AST-root) type formers: for each term former f we freely add a type former $f^\sharp$ whose inhabitants are exactly those terms whose root former is f. Second, at each carrier and sort we add propositional operations on type formers; crucially, we stratify which operators are available by the categorical strength needed to interpret them (finite limits first, then additional structure for $\bot$, finite joins/distributivity, and finally implication/negation).

Contributions. This paper contributes:

- a syntactic construction that generates modal type formers from base rewrites and redex positions (and equation modalities from definitional equalities);
- an empty-context possibility operator $\Diamond$ yielding a rewrite-stable, weakened conversion principle;
- a local-slot / equational-center mechanism that organizes admissible $*/\Box$ choices into a (hyper)cube of calculi stable under equations and rewrites;
- a soundness theorem interpreting generated type formers as comprehension (and image) subobjects in any finite-limits model with a subobject fibration;
- two orthogonal extensions: structural (AST-root) types and stratified propositional operations on type formers;
- a Curry–Howard normalization principle for modalities, matching modal cut elimination with firing a one-step rewrite (and reflecting nondeterminism when present).

Roadmap. Section 3 gives a brief overview. Section 4 defines lambda theories and their internal-language judgments. Section 5 develops the modal layer and the modal hypercube (including modal cut elimination in Subsection 5.9). Sections 6 and 7 add structural and propositional layers.

2 Review of the Lambda Cube

We briefly recall the standard presentation of Barendregt's Lambda Cube as a family of generalized type systems parameterized by which *sort pairs* are permitted in Π-formation and λ-abstraction [4].

Sorts, variables, and raw terms. Let $S := \{*, \Box\}$, with metavariables $s, s_1, s_2 \in S$, and let V be a set of variables $x, y, \ldots$. Raw terms (which serve as both terms and types) are generated by:

$$A \ ::= \ x \ \mid \ s \ \mid \ A\,A \ \mid \ \lambda x : A.\, A \ \mid \ \Pi x : A.\, A.$$

Environments. Typing environments (contexts) are lists of typed variables:

$$\Gamma \ ::= \ \emptyset \ \mid \ \Gamma,\, x : A.$$

β-reduction and conversion. The one-step reduction $\rightsquigarrow_\beta$ is the compatible closure of the β-redex:

$$\frac{}{(\lambda x : A.\, B)\, C \rightsquigarrow_\beta B[C/x]}\ \beta \qquad \frac{A \rightsquigarrow_\beta A'}{A\, B \rightsquigarrow_\beta A'\, B}\ \textsc{App}_1 \qquad \frac{B \rightsquigarrow_\beta B'}{A\, B \rightsquigarrow_\beta A\, B'}\ \textsc{App}_2$$

$$\frac{A \rightsquigarrow_\beta A'}{\lambda x : A.\, B \rightsquigarrow_\beta \lambda x : A'.\, B}\ \textsc{Lam}_1 \qquad \frac{B \rightsquigarrow_\beta B'}{\lambda x : A.\, B \rightsquigarrow_\beta \lambda x : A.\, B'}\ \textsc{Lam}_2$$

$$\frac{A \rightsquigarrow_\beta A'}{\Pi x : A.\, B \rightsquigarrow_\beta \Pi x : A'.\, B}\ \textsc{Pi}_1 \qquad \frac{B \rightsquigarrow_\beta B'}{\Pi x : A.\, B \rightsquigarrow_\beta \Pi x : A.\, B'}\ \textsc{Pi}_2$$

Let $=_\beta$ denote β-convertibility (the reflexive, transitive, symmetric closure of $\rightsquigarrow_\beta$).

Typing judgment and common rules. The typing judgment is $\Gamma \vdash A : B$. The following rules are common to every system in the cube:

$$\frac{}{\vdash * : \Box}\ \textsc{Axiom} \qquad \frac{\Gamma \vdash A : s}{\Gamma, x : A \vdash x : A}\ \textsc{Start} \quad (x \notin \Gamma)$$

$$\frac{\Gamma \vdash A : B \quad \Gamma \vdash C : s}{\Gamma, x : C \vdash A : B}\ \textsc{Weakening} \quad (x \notin \Gamma) \qquad \frac{\Gamma \vdash C : \Pi x : A.\, B \quad \Gamma \vdash D : A}{\Gamma \vdash C\, D : B[D/x]}\ \textsc{Application}$$

$$\frac{\Gamma \vdash A : B \quad B =_\beta B' \quad \Gamma \vdash B' : s}{\Gamma \vdash A : B'}\ \textsc{Conversion}\ .$$

Product and abstraction (parameterized by sort pairs). Fix a set of allowed sort pairs $R \subseteq S \times S$. The remaining rules are:

$$\frac{\Gamma \vdash A : s_1 \quad \Gamma, x : A \vdash B : s_2 \quad (s_1, s_2) \in R}{\Gamma \vdash \Pi x : A.\, B : s_2}\ \textsc{Product}$$

$$\frac{\Gamma \vdash A : s_1 \quad \Gamma, x : A \vdash B : s_2 \quad \Gamma, x : A \vdash C : B \quad (s_1, s_2) \in R}{\Gamma \vdash \lambda x : A.\, C : \Pi x : A.\, B}\ \textsc{Abstraction}\ .$$

The cube (eight choices of R). All eight systems include $(*, *)$ (terms depending on terms). The remaining three pairs correspond to the three additional dependency directions:

$(*, \Box)$ (types depending on terms),

$(\Box, *)$ (terms depending on types),

$(\Box, \Box)$ (types depending on types).

Choosing any subset of these three pairs (together with $(*,*)$) yields one vertex of the cube:

	$(*,*)$	$(*,\Box)$	$(\Box,*)$	$(\Box,\Box)$
$\lambda_{\to}$	✓			
λP	✓	✓		
$\lambda 2$	✓		✓	
$\lambda\underline{\omega}$	✓			✓
$\lambda P2$	✓	✓	✓	
$\lambda P\underline{\omega}$	✓	✓		✓
$\lambda\omega$	✓		✓	✓
λC	✓	✓	✓	✓

3 Overview of the Hypercube Construction

This section is purely expository: it summarizes the construction and the role of each major ingredient before the formal definitions.

3.1 Input: operational data in a lambda theory

We begin with a lambda theory T: a category with finite limits and a chosen cartesian closed structure in whose internal language we can state equations, operations, and a one-step reduction relation $\rightsquigarrow \subseteq \mathrm{Pr} \times \mathrm{Pr}$ on a distinguished object of processes Pr. Operational rules appear as *base rewrites* of the form

$$x_1 : X_1, \ldots, x_n : X_n \mid \emptyset \vdash L(\vec{x}) \rightsquigarrow R(\vec{x}).$$

Crucially, we also track *redex positions*: choosing a subterm occurrence t_j of L determines a one-hole context $K_j[-]$ with $K_j[t_j] = L$, and the free variables of K_j determine the rely parameters of the modality generated at that position.

3.2 Output: three layers of type formers

From T we freely generate a family of type systems organized as a hypercube:

- **Modal layer (Section 5).** Each base rewrite and each subterm position on the left-hand side produces a modality $\langle K_j \rangle_{\vec{x}::\vec{A}} B$, read as a *rely-possibly* specification: under rely assumptions $\vec{x} :: \vec{A}$, placing a term in context $K_j[-]$ steps to a specified reduct of type B. Equation-generated modalities are treated similarly (as "zero-step" steps), but their eliminations admit full conversion. To express a rewrite-stable, existential one-step possibility, we also generate an empty-context operator $\Diamond$.
- **Structural layer (Section 6).** Each term former f produces a structural type former $f^\sharp$ that classifies terms by the root of the parse tree.
- **Propositional layer (Section 7).** Each carrier and sort gets propositional operations on type formers, stratified by the categorical structure available on the base theory (finite limits first, then additional structure for the remaining connectives).

The modal layer is designed to be a small, Cube-like core; the other layers increase expressivity in a controlled way.

3.3 Local sort slots and the equational center

Each generated type former comes with finitely many *sort slots* (one per rely input and one for the output). Filling each slot with $*$ or $\square$ yields a raw Boolean cube of choices. However, equational axioms and rewrite laws force certain slots to agree: only the assignments stable under these laws survive, forming the *equational center* $\mathcal{Z}$. Vertices of the final hypercube are the admissible global sort assignments in $\mathcal{Z}$.

[[TODO: Mention that Barendregt didn't need to do this because there are no built-in equations and the underlying equations of the rewrites don't add any relations between sorts.]]

3.4 TODO: Lambda calc example?

4 Lambda Theories

We follow the usage of *lambda theory* from the setting of equational logics with binding and rewriting [**?**]: a calculus is presented by types and terms (the objects and morphisms of a base category), propositions about them (via the subobject fibration), and a distinguished one-step rewrite predicate on programs.

4.1 Lambda theories and rewriting

We write $\pi : \mathrm{Sub}(\mathrm{T}) \to \mathrm{T}$ for the subobject fibration[8, 9, 10]: an object over Γ is a mono $\varphi \rightarrowtail \Gamma$ (a predicate in context Γ), entailment is inclusion, and substitution is pullback. In internal-language terms, for each relation symbol $R : X_1 \times \cdots \times X_n \to \mathrm{prop}$ and terms $\Gamma \vdash t_i : X_i$ we have an atomic proposition $R(t_1, \ldots, t_n)$, written $\Gamma \vdash R(t_1, \ldots, t_n)\ \mathrm{prop}$.

Definition 4.1 (Lambda theory)**.** A **lambda theory** consists of:

- a category T with finite limits and a chosen cartesian closed structure[5, 2];
- its subobject fibration $\pi : \mathrm{Sub}(\mathrm{T}) \to \mathrm{T}$ (the ambient logic);
- a distinguished object $\mathrm{Pr} \in \mathrm{T}$ of programs/processes;
- a distinguished subobject (proposition)

$$(\rightsquigarrow) \ \rightarrowtail\ \mathrm{Pr} \times \mathrm{Pr},$$

  written in the internal language as $p : \mathrm{Pr},\ q : \mathrm{Pr} \vdash (p \rightsquigarrow q)\,\mathrm{prop}$;
- a specified family of entailments generating and closing rewriting, consisting of:
  - **base rewrites** $\rightsquigarrow_B$ of the form

$$\Gamma \mid \emptyset \ \vdash\ L(\vec{x}) \rightsquigarrow R(\vec{x}) \qquad \text{with} \qquad \Gamma \vdash L(\vec{x}) : \mathrm{Pr},\ \Gamma \vdash R(\vec{x}) : \mathrm{Pr}.$$

  - **rewrite operations** $\rightsquigarrow_O$, expressing closure under the designated term formers, of the schematic form

$$\Gamma \mid \Big(\bigwedge_{i \in I} (p_i \rightsquigarrow q_i)\Big) \ \vdash\ f(\vec{p}) \rightsquigarrow f(\vec{q}),$$

    where f ranges over the operations in the presentation, I ranges over a designated subset of argument positions, and the remaining arguments are required (by equations or additional hypotheses) to be equal.

4.2 Example: The Lambda Theory of the ρ-Calculus

We briefly present the lambda theory underlying the ρ-calculus[13, 12]. The ρ-calculus is a reflective higher-order (RHO) polyadic asynchronous pi calculus[14] with two primary types:

- a type of processes Pr,
- a type of names Nm,

and a distinguished rewrite relation $(\rightsquigarrow) \subseteq \mathrm{Pr} \times \mathrm{Pr}$ capturing one-step computational steps.

In the $\rho-$calculus, names are not created afresh with the ν operator; instead, each name is the serialization of a process. There are two operators for serialization and deserialization, @ and $*$, respectively. The $\rho-$calculus also has no replication, since it can be simulated using these operators. When sending a process Q, it is serialized before allowing any rewrites of Q to occur. The receiving process gets the serialization $@Q$.

Base types and operations. The base types are

$$\mathrm{T}_{\mathrm{type}} = \{\mathrm{Pr}, \mathrm{Nm}\}.$$

The process and name formers are:

$$\begin{aligned}
\vdash\ & 0 : \mathrm{Pr} \\
p_1,\ p_2 : \mathrm{Pr}\ \vdash\ & p_1 \mid p_2 : \mathrm{Pr} \\
p : \mathrm{Pr}\ \vdash\ & @p : \mathrm{Nm} \\
n : \mathrm{Nm}\ \vdash\ & *n : \mathrm{Pr} \\
\forall k \in \mathbb{N}. n : \mathrm{Nm},\ \vec{p} : \mathrm{Pr}^k\ \vdash\ & \mathsf{out}_k(n, \vec{p}) : \mathrm{Pr} \\
\forall k \in \mathbb{N}. n : \mathrm{Nm},\ \lambda\vec{x}.\, q : [\mathrm{Nm}^k \to \mathrm{Pr}]\ \vdash\ & \mathsf{in}_k(n, \lambda\vec{x}.q) : \mathrm{Pr},
\end{aligned}$$

where $[\mathrm{Nm}^k \to \mathrm{Pr}]$ is the exponential type of k-ary name-indexed process continuations.

Equations. Processes form a commutative monoid under juxtaposition $-|-$ with unit 0:

$$\begin{aligned}
p : \mathrm{Pr}\ \vdash\ & p \mid 0 = p \\
p_1, p_2 : \mathrm{Pr}\ \vdash\ & p_1 \mid p_2 = p_2 \mid p_1 \\
p_1, p_2, p_3 : \mathrm{Pr}\ \vdash\ & (p_1 \mid p_2) \mid p_3 = p_1 \mid (p_2 \mid p_3).
\end{aligned}$$

Reflection between names and processes satisfies:

$$n : \mathrm{Nm}\ \vdash\ @(*n) = n.$$

Rewrite proposition and base rewrites. We take the distinguished rewrite proposition as

$$p_1 : \mathrm{Pr},\ p_2 : \mathrm{Pr}\ \vdash\ (p_1 \rightsquigarrow p_2)\ \mathrm{prop}.$$

The base rewrites $\rightsquigarrow_B$ are:

- *Communication.* For each arity k,

$$n : \mathrm{Nm},\ \vec{p} : \mathrm{Pr}^k,\ \lambda\vec{x}.\, q : [\mathrm{Nm}^k \to \mathrm{Pr}]\ \vdash\ \mathsf{out}_k(n, \vec{p}) \mid \mathsf{in}_k(n, \lambda\vec{x}.q)\ \rightsquigarrow\ q[@\vec{p}/\vec{x}].$$

- *Reflection round-trip.*

$$p : \mathrm{Pr}\ \vdash\ *(@p) \rightsquigarrow p.$$

Here $q[@\vec{p}/\vec{x}]$ denotes the simultaneous substitution of the quoted processes $@p_i$ for the bound name variables x_i.

Rewrite operations (congruence for parallel). Rewrite operations $\rightsquigarrow_O$ close the base rewrites under juxtaposition. In the ρ-calculus we allow rewriting in either or both arguments to juxtaposition:

$$p_1, p_2, q : \mathrm{Pr} \mid (p_1 \rightsquigarrow p_2) \vdash p_1 \mid q \rightsquigarrow p_2 \mid q,$$
$$p_1, p_2, q_1, q_2 : \mathrm{Pr} \mid (p_1 \rightsquigarrow p_2),\ (q_1 \rightsquigarrow q_2) \vdash p_1 \mid q_1 \rightsquigarrow p_2 \mid q_2.$$

Thus the rewrite relation is compatible with the commutative monoid structure on Pr.

Lambda-theory summary. Collecting the clauses above, we obtain a presentation $P(\mathrm{T}_\rho)$ of a lambda theory T_ρ with:

- base types Pr, Nm,
- operations 0, $|$, $@$, $*$, out_k, in_k,
- equations making $(\mathrm{Pr}, |, 0)$ a commutative monoid and giving the round-trip law $n : \mathrm{Nm} \vdash @(*n) = n$,
- a rewrite proposition $(\rightsquigarrow) \hookrightarrow \mathrm{Pr} \times \mathrm{Pr}$ with base rewrites for communication and reflection and rewrite operations closing these under juxtaposition.

4.3 Comparison of Lambda theories to other formalizations of operational semantics

TODO

5 Modal Types and the Minimal Hypercube

In this section we define the modal layer: per-object sort structure, modal types derived from base rewrites (and from equations), the local sort slots attached to each modality, and the *equational center* that constrains sort assignments from the outset. Equations and base rewrites of the underlying lambda theory are always present; they induce algebraic constraints on sort assignments through the equational center, so that sorts are well behaved on the quotient of the syntax by both equations and operational laws. Throughout, we single out the modal layer as a minimal, Cube-like core; richer structural and propositional structure on type formers is added later in Sections 6 and 7.

5.1 Per-Object Enrichment

For each object X of T, freely add distinguished *sort constants*

$$\vdash *^X : X, \qquad \vdash \square^X : X,$$

and a typing/kinding relation symbol

$$x : X,\ y : X \vdash \big(x ::^X y\big) \text{ prop},$$

together with the *sort axiom*

$$\vdash *^X ::^X \square^X.$$

Convention. We write $x ::^X y$ to denote the typing relation at carrier X; when X is clear from context, we suppress the superscript and write $x :: y$.

The distinguished elements $*^X$ and $\Box^X$ play the role of *sort symbols* at carrier X. We think of $A :: *^X$ as saying that A is a *type former* at X, and of $A :: \Box^X$ as saying that A is a *kind former* at X.

Remark. We do *not* fix any concrete meaning for the constants $*^X$ and $\Box^X$ beyond their role as sort markers. Soundness interprets $A :: *^X$ and $A :: \Box^X$ as predicates in a model (Section 5.7).

5.2 Common typing rules

We work with judgments in two contexts:

$$\Gamma \,|\, \Delta \;\vdash\; \psi.$$

Intuitively, Γ declares variables and their carriers (objects of T), while Δ lists assumed propositions (including typing assumptions $t :: A$ and reduction assumptions $p \rightsquigarrow q$).

Sort axiom.

$$\frac{}{\emptyset \,|\, \emptyset \;\vdash\; *^X ::^X \Box^X}\ \textsc{Sort-Ax}$$

Start.

$$\frac{\Gamma \,|\, \Delta \;\vdash\; A ::^X s^X}{\Gamma,\ x : X \,|\, \Delta,\ x ::^X A \;\vdash\; x ::^X A}\ \textsc{Start}$$

Weakening.

$$\frac{\Gamma \,|\, \Delta \;\vdash\; t ::^Y B \quad \Gamma \,|\, \Delta \;\vdash\; A ::^X s^X}{\Gamma,\ x : X \,|\, \Delta,\ x ::^X A \;\vdash\; t ::^Y B}\ \textsc{Weak}$$

Conversion (definitional equality). Let $=_{\beta\eta}$ denote definitional equality at each carrier, generated by the chosen CCC $\beta\eta$-laws and the equations of T.

$$\frac{\Gamma \,|\, \Delta \;\vdash\; t ::^X A \quad \Gamma \vdash A =_{\beta\eta} A' : X \quad \Gamma \,|\, \Delta \;\vdash\; A' ::^X s^X}{\Gamma \,|\, \Delta \;\vdash\; t ::^X A'}\ \textsc{Conv}$$

5.3 Dependent product

A key point is that in a cartesian closed setting, *equations* (definitional equalities) can be treated as "zero-step" operational data for generating modalities. In particular, the CCC β-law for evaluation yields a genuine dependent product, and all modalities from equations can be represented as a dependent product.

Fix objects X, Y in T and write $X \to Y$ for the exponential. Let $\mathrm{eval} : (X \to Y) \times X \to Y$ be evaluation. Consider the one-holed evaluation context

$$E([-]) \;:=\; \mathrm{eval}([-], x),$$

where $x : X$ is a (bound) rely variable. Define the dependent product type former by

$$\Pi_{x::A} B \;:=\; \langle \mathrm{eval}([\,], x) \rangle_{x::A}\, B,$$

a type former at carrier $X \to Y$.

5.3.1 Formation

$$\frac{\Gamma \,|\, \Delta \;\vdash\; A :: s_1^X \quad \Gamma,\ x : X \,|\, \Delta,\ x :: A \;\vdash\; B :: s_2^Y}{\Gamma \,|\, \Delta \;\vdash\; \Pi_{x::A} B :: s_2^{X \to Y}}\ \Pi\textsc{-Form}$$

5.3.2 Introduction

$$\frac{\Gamma \,\big|\, \Delta \;\vdash\; A :: s_1^X \quad \Gamma,\; x : X \,\big|\, \Delta,\; x :: A \;\vdash\; B :: s_2^Y \quad \Gamma,\; x : X \,\big|\, \Delta,\; x :: A \;\vdash\; C :: B}{\Gamma \,\big|\, \Delta \;\vdash\; (\lambda x.\, C) :: \Pi_{x::A} B} \;\text{Π-Intro}$$

5.3.3 Elimination (application)

$$\frac{\Gamma \,\big|\, \Delta \;\vdash\; f :: \Pi_{x::A} B \quad \Gamma \,\big|\, \Delta \;\vdash\; u :: A}{\Gamma \,\big|\, \Delta \;\vdash\; \mathrm{eval}(f, u) :: B[u/x]} \;\text{Π-Elim}$$

Unlike rewrite-generated modalities (below), equation-generated modalities support full conversion: $\mathrm{eval}(f, u)$ is directly typed at $B[u/x]$ (not merely at $\Diamond(B[u/x])$).

5.4 Modalities from Base Rewrites

Fix a base rewrite

$$x_1 : X_1, \ldots, x_n : X_n \;\mid\; \emptyset \;\vdash\; L(\vec{x}) \;\rightsquigarrow\; R(\vec{x})$$

at carrier Pr. Choose any subterm occurrence $t_j \subseteq L$ with carrier Y_j and one-holed context $K_j[-]$ (so that $K_j[t_j] = L$). Write $V_j = \mathrm{FV}(K_j) - \mathrm{FV}(t_j) \subseteq \{x_1, \ldots, x_n\}$ for the set of variables free in K_j that are not free in t_j and $W_j = \mathrm{FV}(t_j) - \mathrm{FV}(K_j)$.

For concision, write $\vec{x} : \vec{X}$ for the list of variables in V_j and their carriers, and write $\vec{x} :: \vec{A}$ for the corresponding list of rely typings, and similarly write $\vec{y} : \vec{Y}$ for the list of variables in W_j and their carriers.

5.4.1 Formation

The modality $\langle K_j \rangle_{\vec{x}::\vec{A}} B$ is a type former at carrier Y_j. It carries one sort slot per rely input A_k and one output sort slot (shared between B and the modality itself).

$$\frac{\big(\Gamma \,\big|\, \Delta \;\vdash\; A_k :: s_k^{X_k}\big)_{x_k \in V_j} \quad \Gamma,\; \vec{x} : \vec{X} \,\big|\, \Delta,\; \vec{x} :: \vec{A} \;\vdash\; B :: s_{\mathrm{out}}^{\mathrm{Pr}}}{\Gamma \,\big|\, \Delta \;\vdash\; \langle K_j \rangle_{\vec{x}::\vec{A}} B :: s_{\mathrm{out}}^{Y_j}} \;\text{M-Form}$$

5.4.2 Introduction

Introduction types the chosen redex component t_j by demanding that the *specified* right-hand side has the desired type under the relies (and the rewrite preconditions).

$$\frac{\begin{array}{l}\big(\Gamma \,\big|\, \Delta \;\vdash\; A_k :: s_k^{X_k}\big)_{x_k \in V_j} \quad \Gamma,\; \vec{w} : \vec{W},\; \vec{x} : \vec{X} \,\big|\, \Delta,\; \vec{x} :: \vec{A} \;\vdash\; B :: s_{\mathrm{out}}^{\mathrm{Pr}} \\ \Gamma,\; \vec{w} : \vec{W},\; \vec{x} : \vec{X} \,\big|\, \Delta,\; \vec{x} :: \vec{A} \;\vdash\; R(\vec{x}, \vec{w}) :: B\end{array}}{\Gamma,\; \vec{w} : \vec{W} \,\big|\, \Delta \;\vdash\; t_j(\vec{w}) :: \langle K_j \rangle_{\vec{x}::\vec{A}} B} \;\text{M-Intro}$$

5.4.3 Elimination: rewrite vs equation

Since $\langle K_j \rangle_{\vec{x}::\vec{A}} B$ is generated from a *rewrite*, elimination does *not* provide full conversion. Instead, it provides:

1. the operational step to the specified right-hand side, and
2. the typing of that right-hand side.

To state these rules, assume we have concrete rely inhabitants $\vec{u}$:

$$\big(\Gamma \,\big|\, \Delta \;\vdash\; u_k :: A_k\big)_{x_k \in V_j}.$$

Step to the specified RHS.

$$\frac{\Gamma \,\big|\, \Delta \;\vdash\; t :: \langle K_j \rangle_{\vec{x}::\vec{A}} B \quad \big(\Gamma \,\big|\, \Delta \;\vdash\; u_k :: A_k\big)_{x_k \in V_j}}{\Gamma \,\big|\, \Delta \;\vdash\; K_j[t][\vec{u}/\vec{x}] \;\rightsquigarrow\; R(\vec{u})} \;\text{M-Step}$$

Typing of the specified RHS.

$$\frac{\Gamma \,\big|\, \Delta \;\vdash\; t :: \langle K_j \rangle_{\vec{x}::\vec{A}} B \quad \big(\Gamma \,\big|\, \Delta \;\vdash\; u_k :: A_k\big)_{x_k \in V_j}}{\Gamma \,\big|\, \Delta \;\vdash\; R(\vec{u}) :: B[\vec{u}/\vec{x}]} \;\text{M-Elim}_{\rightsquigarrow}$$

Weakened conversion via $\Diamond$. From the step and the typing of the RHS we obtain a weakened typing of the redex-in-context:

$$\Gamma \,\big|\, \Delta \;\vdash\; K_j[t][\vec{u}/\vec{x}] :: \Diamond\big(B[\vec{u}/\vec{x}]\big).$$

This is stated formally in Subsection 5.5.

5.4.4 Rely-Possibly Reading

Fix a context K_j and rely types A_k. Semantically (see 5.7), the type former $\langle K_j \rangle_{\vec{x}::\vec{A}} B$ is interpreted as the comprehension of those $t : Y_j$ that satisfy the internal formula

$$\forall (x_k : X_k)_{x_k \in V_j}.\, \Big(\Big(\bigwedge_{x_k \in V_j} x_k :: [\![A_k]\!]\Big) \;\Rightarrow\; \big([\![K_j]\!](t) \rightsquigarrow [\![R(\vec{x})]\!]\big) \;\wedge\; \big([\![R(\vec{x})]\!] :: [\![B]\!]\big)\Big).$$

This semantics matches the proof rules above: membership in the modality provides (i) a step to the specified right-hand side and (ii) typing information about that right-hand side.

5.5 Empty-context possibility $\Diamond$

To recover a conversion-like principle stable under rewriting, we freely add an *empty-context possibility* operator at carrier Pr:

$$\Diamond(-) : \mathrm{Pr} \to \mathrm{Pr},$$

intended so that $p :: \Diamond B$ asserts that p can take *one* step to some reduct of type B.

5.5.1 Formation

$$\frac{\Gamma \,\big|\, \Delta \;\vdash\; B :: s^{\mathrm{Pr}}}{\Gamma \,\big|\, \Delta \;\vdash\; \Diamond B :: s^{\mathrm{Pr}}} \;\Diamond\text{-Form}$$

5.5.2 Introduction

$$\frac{\Gamma \,\big|\, \Delta \;\vdash\; p \rightsquigarrow q \quad \Gamma \,\big|\, \Delta \;\vdash\; q :: B}{\Gamma \,\big|\, \Delta \;\vdash\; p :: \Diamond B} \;\Diamond\text{-Intro}$$

5.5.3 Elimination

$$\frac{\Gamma \,\big|\, \Delta \;\vdash\; p :: \Diamond B \quad \Gamma,\; q : \mathrm{Pr} \,\big|\, \Delta,\; (p \rightsquigarrow q),\; (q :: B) \;\vdash\; \varphi}{\Gamma \,\big|\, \Delta \;\vdash\; \varphi} \;\Diamond\text{-Elim} \qquad (q \notin \mathrm{FV}(\varphi))$$

5.5.4 Derived lax conversion for rewrite modalities

Combining the rewrite-modality step and reduct-typing rules with $\Diamond$-Intro yields:

$$\frac{\Gamma \,\big|\, \Delta \;\vdash\; t :: \langle K_j \rangle_{\vec{x}::\vec{A}} B \quad \big(\Gamma \,\big|\, \Delta \;\vdash\; u_k :: A_k\big)_{x_k \in V_j}}{\Gamma \,\big|\, \Delta \;\vdash\; K_j[t][\vec{u}/\vec{x}] :: \Diamond\big(B[\vec{u}/\vec{x}]\big)} \;\text{M-ELIM}_{\rightsquigarrow,\Diamond}$$

This is the promised weakened conversion principle: the redex-in-context is typed at $\Diamond(B[\vec{u}/\vec{x}])$, reflecting that it can take one step to a term of type $B[\vec{u}/\vec{x}]$.

5.6 Modality-Local Sort Assignments

The product and abstraction rules in the lambda cube depend on a choice of two sorts s_1 and s_2, one for the input sort and one for the output sort. Similarly, the modality formation and introduction rules have sorts s_k for the rely inputs and a single output sort s_{out} shared between the reduct specification B (at carrier Pr) and the modality itself (at carrier Y_j). Thus, for a context with $m := |V_j|$ rely parameters, a modality contributes $m + 1$ local sort slots.

5.6.1 Example: weak monoids

[[TODO: more interesting example?]]

Here's the lambda theory of a "weak" commutative monoid, where we have replaced the equations for associativity and the unit laws with rewrites that lead to a left-parenthesized, unit-free normal form.

- term formation axioms
  - $\vdash e : \text{Pr}$
  - $p, q : \text{Pr} \vdash (pq) : \text{Pr}$
- a commutativity axiom $p, q : \text{Pr} \vdash (pq) = (qp)$
- rewrite axioms
  - base rewrites
    - **associate:** $p, q, r : \text{Pr} \vdash (p(qr)) \rightsquigarrow ((pq)r)$
    - **unit:** $p : \text{Pr} \vdash (pe) \rightsquigarrow p$
  - context rewrite
    - $p_1, p_2, q : \text{Pr} \mid p_1 \rightsquigarrow p_2 \vdash (p_1 q) \rightsquigarrow (p_2 q)$

We get four modalities for the four proper subterms p, q, r, and (qr) of the left-hand side of the associate rewrite and two modalities for the two proper subterms p and e of the left-hand side of the unit rewrite. Suppose we choose the associate rewrite and p as our subterm, with the context $([](qr))$. The context has free variables q and r, with associated types/kinds A_q and A_r. The formation rule for the corresponding modality says

$$\frac{\Gamma \,\big|\, \Delta \;\vdash\; A_q :: s_q^{\text{Pr}} \quad \Gamma \,\big|\, \Delta \;\vdash\; A_r :: s_r^{\text{Pr}} \quad \Gamma,\; q : \text{Pr},\; r : \text{Pr} \,\big|\, \Delta,\; q :: A_q,\; r :: A_r \;\vdash\; B :: s_{\text{out}}^{\text{Pr}}}{\Gamma \,\big|\, \Delta \;\vdash\; \langle ([](qr)) \rangle_{q::A_q,\; r::A_r} B :: s_{\text{out}}^{\text{Pr}}}$$

Naively, we have a choice of whether to allow $\Box$ or not for each of $s_q^{\text{Pr}}, s_r^{\text{Pr}}$, and $s_{\text{out}}^{\text{Pr}}$, which would lead to a $2^3 - 1 = 7$-dimensional local cube of nontrivial choices for this modality.

But the fact that $(qr) = (rq)$ means that s_q^{Pr} has to equal s_r^{Pr}, and the fact that the reduct type former B is used in an output position means $s_{\text{out}}^{\text{Pr}}$ has to equal s_q^{Pr}, too. In this example, the equational center collapses the naive cube: we have only one degree of freedom instead of three. The hypercube of calculi is a mere line segment with two endpoints corresponding to the simply-typed calculus and the calculus in which one can also multiply types.

5.6.2 General case

Let a base rewrite be

$$x_1 : X_1, \ldots, x_n : X_n \ \vdash \ L(\vec{x}) \ \rightsquigarrow \ R(\vec{x}).$$

For each subterm $t_j \subseteq L$ with carrier Y_j, let K_j be the one-hole context with $K_j[t_j] = L$. Write $V_j \subseteq \{x_1, \ldots, x_n\}$ for the set of variables free in K_j.

Each modality $\langle K_j \rangle_{\vec{x}::\vec{A}} B$ carries its own slot family

$$\mathrm{Slots}(K_j, B) \ := \ \{ s_k^{X_k} \}_{k \in V_j} \ \cup \ \{ s_{\mathrm{out}} \},$$

where s_{out} is a single output coordinate used both as $s_{\mathrm{out}}^{\mathrm{Pr}}$ for B and as $s_{\mathrm{out}}^{Y_j}$ for the modality's own carrier.

For a fixed modality instance, a *raw* local sort assignment is a function

$$\sigma_{K_j,B} : \ \mathrm{Slots}(K_j, B) \ \longrightarrow \ \{*, \Box\}.$$

Different subterms of the same rewrite may yield different local cube dimensions.

We collect all slots across all generated type formers into a finite set Slot, so that a *raw* global sort assignment is a function

$$\Sigma : \mathsf{Slot} \longrightarrow \{*, \Box\}.$$

Equations and rewrites constrain Σ by requiring that sort-level interpretations of equal terms (or rewrite-related terms) agree. The set of admissible assignments forms the *equational center* $\mathcal{Z} \subseteq \{*, \Box\}^{\mathsf{Slot}}$.

Definition 5.1 (Equationally admissible sort assignments and equational center)**.** A raw sort assignment $\Sigma : \mathsf{Slot} \to \{*, \Box\}$ is **equationally admissible** if it preserves sort-level interpretations of all equational axioms and base rewrite laws (and hence their context closures). The **equational center** $\mathcal{Z}$ is the set of all such admissible assignments.

A point $\Sigma \in \mathcal{Z}$ determines, for each modality $\langle K_j \rangle_{\vec{x}::\vec{A}} B$, an admissible local sort assignment by restriction. Taking the product over all base rewrites and all subterm positions yields the *modal hypercube* of calculi.

Computing $\mathcal{Z}$. Since Slot is finite in any finite presentation, $\mathcal{Z}$ is effectively computable by brute-force enumeration and filtering.

5.7 Soundness for the Modal Layer

We briefly sketch the semantics of the modal layer in a model of T, postponing the details of propositional and structural type formers to Sections 6 and 7.

Fix a model M of T in a finite-limits category. We interpret:

- The carrier objects of T as objects of M, writing X both for an object of T and its image in M. [[TODO: replace by explicit notation for the model.]]
- The typing/kinding relation :: as a subobject of $X \times X$, and the reduction relation $\rightsquigarrow$ as a subobject of $\mathrm{Pr} \times \mathrm{Pr}$. The distinguished elements $*^X, \Box^X : 1 \to X$ then determine predicates on X expressing "is a type former at X" and "is a kind former at X".
- For each carrier X and each sort symbol $s^X \in \{*^X, \Box^X\}$, the *universe of term formers of sort* s^X as the subobject

$$U_{s^X} \hookrightarrow X$$

(when the ambient logic supports it) by comprehending the internal formula

$$A :: s^X.$$

(Equivalently: U_{s^X} is the subobject cut out by the predicate $(-) :: s^X$.)

- Each type former A at carrier X and sort s^X (i.e. $A :: s^X$) as a subobject

$$[\![A]\!] \hookrightarrow X$$

given by comprehension of the predicate $t :: A$ on X.

- Each rewrite modality former $\langle K_j \rangle_{\vec{x}::\vec{A}} B : Y_j$ as the comprehension subobject of Y_j whose elements $t : Y_j$ satisfy the rely-possibly formula in Section 5.4.4.

- The empty-context possibility $\Diamond B$ as the comprehension subobject of Pr defined (when the ambient logic supports $\exists$) by:

$$p :: \Diamond B \quad \equiv \quad \exists q : \mathrm{Pr}.\ (p \rightsquigarrow q) \wedge (q :: B).$$

Under this interpretation, the formation, introduction, and elimination rules for modal types and $\Diamond$ are validated by the usual introduction and elimination principles for comprehension subobjects in the internal logic of M. Every rule instance of the modal fragment is valid in every such model; the modal layer is sound.

5.8 Dependent Product and Barendregt's Lambda Cube

The raw modal hypercube construction is already visible in the familiar case of dependent products: the equation modality induced by evaluation yields Π and the allowed sort pairs recover Barendregt's organization.

In contrast, if one presents β as a rewrite (rather than definitional equality), then the corresponding rewrite-generated modality yields a lax application principle via $\Diamond$. This highlights the role of $\Diamond$ as a rewrite-stable weakening of conversion.

5.9 Modal Cut Elimination and One-Step Rewrites

In the standard Curry–Howard correspondence, cut elimination for implications and dependent products matches β-reduction of λ-terms: a derivation that first introduces a function type and then immediately eliminates it contains a *detour*, and normalizing that detour is exactly the underlying computational step.

In the present setting, the same pattern appears at the modal layer. Each rewrite modality

$$\langle K_j \rangle_{\vec{x}::\vec{A}} B : Y_j$$

is generated from a base rewrite and a choice of subterm position on its left-hand side. Its introduction and elimination rules are designed so that:

- introduction packages typing of the specified one-step reduct $R(\vec{x}) :: B$ under relies and preconditions;

- elimination unpacks this into (i) a concrete step to $R(\vec{u})$ and (ii) typing of $R(\vec{u})$; and via $\Diamond$, (iii) a weakened typing of the redex-in-context.

Normalizing an introduction-followed-by-elimination detour corresponds precisely to firing the underlying rewrite at the chosen redex position.

We use $\mathcal{D}$ and $\mathcal{E}$ as metavariables for *derivation trees*. A *modal cut redex* is a composite derivation in which a modality is introduced and then immediately used to derive a $\Diamond$-fact about a redex-in-context, which is then eliminated:

$$\dfrac{\dfrac{\dfrac{\mathcal{D}}{\Gamma \,\big|\, \Delta \;\vdash\; t :: \langle K_j \rangle_{\vec{x}::\vec{A}}\, B}\;\textsc{M-intro} \qquad \text{(rely inhabitants and preconditions)}}{\Gamma \,\big|\, \Delta \;\vdash\; K_j[t] :: \Diamond B}\;\textsc{M-elim}_{\rightsquigarrow,\Diamond} \qquad \mathcal{E}}{\Gamma \,\big|\, \Delta \;\vdash\; \psi}\;\Diamond\text{-elim}\,.$$

A single *modal* cut-elimination step inlines the concrete reduct forced by the underlying rewrite into the open derivation $\mathcal{E}$. Operationally, this corresponds to firing the one-step rewrite at the chosen redex position.

Unlike λ-calculus β-reduction, ρ-calculus reduction is inherently nondeterministic: a process may have several distinct one-step reducts, and parallel composition introduces genuine races. Proof-theoretically, this means that modal cut elimination need not be confluent.

For example, consider a single output racing between two matching inputs:

$$P \;:=\; \mathsf{out}_1(n, p) \mid \mathsf{in}_1(n, \lambda x.\, q) \mid \mathsf{in}_1(n, \lambda y.\, r).$$

There are two distinct one-step reductions, depending on which input synchronizes:

$$P \rightsquigarrow P_1 := q[@p/x] \mid \mathsf{in}_1(n, \lambda y.\, r), \qquad P \rightsquigarrow P_2 := r[@p/y] \mid \mathsf{in}_1(n, \lambda x.\, q).$$

In the modal system, judgments about $\Diamond$ can be proved by exhibiting a concrete one-step reduct in $\Diamond$-Intro. Thus the operational nondeterminism is reflected as the existence of multiple distinct introduction derivations of the same $\Diamond$-conclusion. Composing either with a fixed $\Diamond$-elimination derivation yields different normal forms, mirroring the operational race.

5.10 Action on Morphisms of Theories

For a morphism $F : \mathrm{T} \to \mathrm{T}'$,

$$*^X \mapsto *^{F(X)}, \quad \square^X \mapsto \square^{F(X)}, \quad (::) \mapsto (::^{F(X)}),$$

and

$$\langle K_j \rangle_{\vec{x}::\vec{A}}\, B \mapsto \langle F(K_j) \rangle_{\vec{x}::F(\vec{A})}\, F{\cdot}B, \qquad \Diamond B \mapsto \Diamond(F{\cdot}B).$$

Thus the modal layer defines, on objects, an endofunctor on the category of lambda theories, and the assignment is natural in morphisms.

Let $\mathbf{L}$ denote the category of lambda theories (with morphisms preserving Pr, ($\rightsquigarrow$), and the internal-language structure), and let $\mathbf{L}_\Sigma$ denote the category of lambda theories equipped with the additional Σ-structure provided by the modal/structural/propositional layer (modalities $\langle K_j \rangle_{\vec{x}::\vec{A}}\, B$, $\Diamond$, structural formers $f^\sharp$, and the chosen propositional operations on type formers).

There is a forgetful functor

$$U : \mathbf{L}_\Sigma \longrightarrow \mathbf{L}$$

that sends a Σ-structured lambda theory to its underlying lambda theory, and a left adjoint

$$F : \mathbf{L} \longrightarrow \mathbf{L}_\Sigma$$

that freely adjoins the Σ-structure to a given lambda theory. Under standard smallness and algebraicity assumptions on presentations, F is left adjoint to U, and the endofunctor

$$\mathcal{H}_\Sigma \;=\; U \circ F : \mathbf{L} \longrightarrow \mathbf{L}$$

is the induced monad on $\mathbf{L}$: for each lambda theory T, $\mathcal{H}_\Sigma(\mathrm{T})$ is the underlying lambda theory of its free Σ-extension.

5.11 Modal types may not type all terms

The modal fragment is intentionally *behavioral*: a judgment $\Gamma \,\big|\, \Delta \;\vdash\; t :: \langle K\rangle_{\vec{x}::\vec{A}}\, B$ is proved by establishing the typing of the specified reduct for a chosen operational rule and redex position. Consequently, terms with no available one-step behavior in any of the generating redex positions need not have *any* modal type in the purely modal fragment.

For this reason we add *structural* type formers in Section 6. These classify terms by their syntactic formers (AST roots), providing a baseline typing for programs, which can then be combined with the modal layer to express both *shape* and *one-step behavior* in the same internal language.

5.12 Example: a $\rho-$calculus modality

The base rewrite and the chosen splitting. Fix arity $k = 1$. The communication rewrite is

$$n : \mathrm{Nm},\; p : \mathrm{Pr},\; \lambda x.\, q : [\mathrm{Nm} \to \mathrm{Pr}] \;\mid\; \emptyset \;\vdash\; \mathsf{out}_1(n, p) \mid \mathsf{in}_1(n, \lambda x.\, q) \;\rightsquigarrow\; q[@p/x].$$

We choose the continuation redex position

$$t_j \;:=\; \lambda x.\, q, \qquad K_j([-]) \;:=\; \mathsf{out}_1(n, p) \mid \mathsf{in}_1(n, [-]),$$

so that $K_j[t_j]$ is equal to the left-hand side. The free variables of K_j are $V_j = \{n, p\}$, hence the induced modality is

$$\langle K_j\rangle_{n::A_n,\; p::A_p}\, B$$

at carrier $[\mathrm{Nm} \to \mathrm{Pr}]$.

The three local sort slots. This modality contributes three local sort slots

$$\mathrm{Slots}(K_j, B) \;=\; \{\, s_n^{\mathrm{Nm}},\; s_p^{\mathrm{Pr}},\; s_{\mathrm{out}} \,\},$$

where s_n^{Nm} sorts the rely former A_n at Nm, s_p^{Pr} sorts the rely former A_p at Pr, and s_{out} is shared between the postcondition $B :: s_{\mathrm{out}}^{\mathrm{Pr}}$ and the modality itself $\langle K_j\rangle_{n::A_n,\; p::A_p}\, B :: s_{\mathrm{out}}^{[\mathrm{Nm}\to\mathrm{Pr}]}$. Relative to the origin assignment $(*, *, *)$, every nonempty subset of these three slots can be lifted to $\square$, yielding $2^3 - 1 = 7$ local axes.

The lambda-cube-like part (s_p, s_{out})**.** For this particular redex position, there is no essential interaction between the communicating (message, continuation) pair and the specific channel n: operationally, n serves only to ensure that the input and output are co-addressed, while the reduct $q[@p/x]$ is determined by substituting the payload p into the continuation q and does not otherwise depend on n. This makes it natural to first treat the two-slot subgeometry governed by (s_p, s_{out}), keeping $s_n = *$ fixed.

With $s_n = *$, the local choices for (s_p, s_{out}) form the familiar "square" whose three nontrivial directions correspond exactly to the three extra edges of Barendregt's Cube beyond $(*, *)$:

Cube-like vertex (at $s_n = *$)	s_p^{Pr}	s_{out}
simply-typed corner	$*$	$*$
polymorphism direction	$\square$	$*$
dependent-types direction	$*$	$\square$
type-constructors direction	$\square$	$\square$

Equivalently, the three *Cube-like axes* inside the face $s_n = *$ are:

Axis p: polymorphism $(\Box, *)$ on (s_p, s_{out}).
Here

$$A_n :: *^{\text{Nm}}, \qquad A_p :: \Box^{\text{Pr}}, \qquad B :: *^{\text{Pr}}.$$

This is the local analogue of the Cube's polymorphism edge: the $*$-level behavioral postcondition is allowed to be indexed by a $\Box$-level payload classifier.

Axis B: dependent types $(*, \Box)$ on (s_p, s_{out}).
Here

$$A_n :: *^{\text{Nm}}, \qquad A_p :: *^{\text{Pr}}, \qquad B :: \Box^{\text{Pr}},$$

and therefore the modality itself is $\Box$-sorted at $[\text{Nm} \to \text{Pr}]$. This matches the Cube's dependent-types edge: a term-level (i.e. $*$-level) input classifier supports a kind-level ($\Box$-level) postcondition/specification.

Axis pB: type constructors $(\Box, \Box)$ on (s_p, s_{out}).
Here

$$A_n :: *^{\text{Nm}}, \qquad A_p :: \Box^{\text{Pr}}, \qquad B :: \Box^{\text{Pr}},$$

and again the modality is $\Box$-sorted. This is the local analogue of the Cube's type-constructor edge: both the payload classifier and the postcondition live at $\Box$.

These three axes account for 3 of the 7 total: they are exactly what you would see if the modality had only one rely parameter (the payload) and one output slot, i.e. the same two-sort shape as Π in the Lambda Cube.

The name sort $s_n = \Box$. Turning on the third coordinate s_n adds a new set of orthogonal axes: it upgrades the *channel-side* rely classifier A_n to be kind-level at carrier Nm.

This matters because the carrier Nm contains not only the original "runtime" names (quotes $@p$) but also newly adjoined Nm-elements such as $*^{\text{Nm}}$ and $\Box^{\text{Nm}}$ (and any further Nm-formers generated by the additional layers). Hence allowing $A_n :: \Box^{\text{Nm}}$ permits channel terms n that are classified using this higher-order Nm-structure. In particular, choosing $A_n := *^{\text{Nm}}$ makes the rely assumption $n :: *^{\text{Nm}}$ assert that the channel term n is itself a *name-type former*.

From the origin $(s_n, s_p, s_{\text{out}}) = (*, *, *)$, the four axes involving the new coordinate s_n are precisely:

axis	s_n^{Nm}	s_p^{Pr}	s_{out}
n	$\Box$	$*$	$*$
np	$\Box$	$\Box$	$*$
nB	$\Box$	$*$	$\Box$
npB	$\Box$	$\Box$	$\Box$

and they can be read uniformly as "the channel-kinding axis n, optionally combined with one of the three Cube-like directions on (s_p, s_{out})":

6 Structural Types from Term Formers

The modal layer classifies one-step behaviors in reduction contexts. Orthogonally, we can add *spatial* (or *structural*) types that classify terms by the shape of their AST at the root.

6.1 Structural Type Formers

Let

$$x_1 : X_1, \ldots, x_n : X_n \;\vdash\; f(x_1, \ldots, x_n) : Y$$

be an n-ary operation in T_{oper}. We introduce a new type former

$$f^\sharp(-_1, \ldots, -_n)$$

at carrier Y. Intuitively, $f^\sharp(A_1, \ldots, A_n)$ is the type of those $y : Y$ that are *literally* built as $f(t_1, \ldots, t_n)$ with $t_i :: A_i$.

Each instance $f^\sharp(A_1, \ldots, A_n)$ carries a slot family

$$\mathrm{Slots}_f(A_1, \ldots, A_n) \;:=\; \{\, s_i^{X_i} \,\}_{i=1}^n \cup \{s_{f,\mathrm{out}}\},$$

consisting of one sort slot for each argument carrier X_i and a single output slot $s_{f,\mathrm{out}}$ (used at carrier Y).

6.2 Typing Rules for Structural Types

We present the rules in the internal language using the compact judgment form $\Gamma \,|\, \Delta \;\vdash\; -$.

Formation.

$$\frac{\big(\Gamma \,|\, \Delta \;\vdash\; A_i :: s_i^{X_i}\big)_{i=1}^n}{\Gamma \,|\, \Delta \;\vdash\; f^\sharp(A_1, \ldots, A_n) :: s_{f,\mathrm{out}}^{Y}} \;\text{S-Form}$$

Introduction.

$$\frac{\big(\Gamma \,|\, \Delta \;\vdash\; A_i :: s_i^{X_i}\big)_{i=1}^n \quad \big(\Gamma \,|\, \Delta \;\vdash\; t_i :: A_i\big)_{i=1}^n}{\Gamma \,|\, \Delta \;\vdash\; f(t_1, \ldots, t_n) :: f^\sharp(A_1, \ldots, A_n)} \;\text{S-Intro}$$

Elimination.

$$\frac{\Gamma,\ (x_i : X_i)_{i=1}^n \,|\, \Delta,\ (x_i :: A_i)_{i=1}^n \;\vdash\; \psi\big(f(x_1, \ldots, x_n)\big)}{\Gamma,\ y : Y \,|\, \Delta,\ y :: f^\sharp(A_1, \ldots, A_n) \;\vdash\; \psi(y)} \;\text{S-Elim}$$

6.3 Slot Families and Equational Center for Structural Types

The general machinery of Subsection 5.4 applies equally to the structural type formers.

6.4 Semantics and Soundness for Structural Types

In a model M of T as in Subsection 5.7, each operation symbol

$$f : X_1 \times \cdots \times X_n \to Y$$

is interpreted as an arrow $f_M : X_1 \times \cdots \times X_n \to Y$, and each type former $A_i :: s_i^{X_i}$ as a subobject $[\![A_i]\!] \hookrightarrow X_i$.

We interpret the structural type

$$f^\sharp(A_1, \ldots, A_n)$$

as the image subobject of the composite

$$[\![A_1]\!] \times \cdots \times [\![A_n]\!] \xrightarrow{m_1 \times \cdots \times m_n} X_1 \times \cdots \times X_n \xrightarrow{f_M} Y.$$

7 Logical/Propositional Operators on Types

The modal and structural layers generate type formers that classify operational behavior and AST shape. In this section we add propositional operations on type formers at each carrier X and each sort symbol $s^X \in \{*^X, \Box^X\}$, but we do so *stratified by categorical strength.* The reason is semantic: for a general finite-limits theory, each fiber $\mathrm{Sub}(X)$ is always a *meet-semilattice with top* (hence supports $\wedge$ and $\top$), but it need not have a bottom element, finite joins, distributivity, or Heyting implication. We therefore split the section into:

- a core fragment valid in every finite-limits theory (top and meet), and
- an extension fragment that adds further constants and connectives when the theory supports them.

7.1 Finite-limits fragment: $\top$ and $\wedge$

Fix an object X of T and a sort symbol $s^X \in \{*^X, \square^X\}$. For any type formers $A, B : X$ at carrier X of sort s^X (i.e. $A :: s^X$ and $B :: s^X$), we freely add in this fragment a meet former

$$A \wedge B : X,$$

and a nullary top former

$$\vdash \top^{s^X} : X.$$

Formation (sort preservation).

$$\frac{\Gamma \,|\, \Delta \;\vdash\; A :: s^X \quad \Gamma \,|\, \Delta \;\vdash\; B :: s^X}{\Gamma \,|\, \Delta \;\vdash\; A \wedge B :: s^X}\ \wedge\text{-FORM} \qquad \frac{}{\Gamma \,|\, \Delta \;\vdash\; \top^{s^X} :: s^X}\ \top\text{-FORM}.$$

Membership rules.

$$\frac{\Gamma \,|\, \Delta \;\vdash\; t :: A \quad \Gamma \,|\, \Delta \;\vdash\; t :: B}{\Gamma \,|\, \Delta \;\vdash\; t :: A \wedge B}\ \wedge\text{-I}.$$

$$\frac{\Gamma \,|\, \Delta \;\vdash\; t :: A \wedge B}{\Gamma \,|\, \Delta \;\vdash\; t :: A}\ \wedge\text{-E}_1 \qquad \frac{\Gamma \,|\, \Delta \;\vdash\; t :: A \wedge B}{\Gamma \,|\, \Delta \;\vdash\; t :: B}\ \wedge\text{-E}_2.$$

Top classifies all terms of the fixed sort s^X; since we do not carry a separate judgment $t \in U_{s^X}$, we introduce $t :: \top^{s^X}$ from the fact that t has *some* type former of that sort:

$$\frac{\Gamma \,|\, \Delta \;\vdash\; A :: s^X \quad \Gamma \,|\, \Delta \;\vdash\; t :: A}{\Gamma \,|\, \Delta \;\vdash\; t :: \top^{s^X}}\ \top\text{-I}.$$

Soundness (finite limits)

Continue with the semantic setup of Subsection 5.7. Fix a carrier X and sort $s^X \in \{*^X, \square^X\}$. In any finite-limits model, each fiber $\mathrm{Sub}(X)$ has:

- a top element $X \hookrightarrow X$, and
- binary meets given by pullback (intersection of subobjects).

We interpret

$$[\![\top^{s^X}]\!] = X, \qquad [\![A \wedge B]\!] = [\![A]\!] \wedge [\![B]\!].$$

With these interpretations, the rules above are sound.

Lemma 7.1 (Meet-semilattice laws)**.** *For each carrier X of* T *and sort symbol $s^X \in \{*^X, \square^X\}$, the collection of type formers $A :: s^X$, modulo mutual entailment*

$$A \simeq B \quad \textit{iff} \quad \vdash \forall t : X.\, (t :: A \Leftrightarrow t :: B),$$

is a meet-semilattice with top under $\wedge$ and $\top^{s^X}$. In particular, up to $\simeq$, $\wedge$ is associative, commutative, and idempotent, and $A \wedge \top^{s^X} \simeq A$.

Sketch. In any finite-limits model, subobjects of X form a meet-semilattice with top; the claims are the standard lattice laws for pullback-intersection and the identity mono. $\square$

7.2 Additional structure: $\bot$, $\vee$, and $\Rightarrow$

We now describe additional constants and connectives that can be added on type formers when the underlying theory has enough structure to interpret the corresponding operations on subobjects.

7.2.1 Adding $\bot$: empty types and modal negation

Extra structure needed. To interpret a bottom element in each fiber $\mathrm{Sub}(X)$, it suffices (and is common) to assume that T has a *strict* initial object 0 and that pullback preserves the initial object. Then the unique arrow $0 \to X$ is monic and gives the least subobject of X.

Syntax and rules. When this structure is available, for each X and s^X we freely add a nullary type former

$$\vdash \bot^{s^X} : X \qquad \text{with} \qquad \vdash \bot^{s^X} :: s^X.$$

There is no introduction rule. Bottom eliminates into any proposition ψ of the ambient internal logic:

$$\frac{\Gamma \,|\, \Delta \;\vdash\; t :: \bot^{s^X}}{\Gamma \,|\, \Delta \;\vdash\; \psi}\ \bot\text{-E}\ .$$

Modal negation (rely-inconsistency). Once $\bot^{s^{\mathrm{Pr}}}$ exists at carrier Pr, we can form the modality

$$\langle K_j \rangle_{\vec{x}::\vec{A}} \bot^{s^{\mathrm{Pr}}}.$$

Unfolding the rely-possibly reading in Section 5.4.4, membership $t :: \langle K_j \rangle_{\vec{x}::\vec{A}} \bot^{s^{\mathrm{Pr}}}$ means that under any parameter assignment satisfying the rely assumptions (and preconditions), the specified right-hand side inhabits $\bot^{s^{\mathrm{Pr}}}$, which forces inconsistency. This is a modal way to express "no successful behavior under the relies".

7.2.2 Adding $\vee$: finite joins and distributivity

Extra structure needed. To interpret disjunction/union, one needs that each $\mathrm{Sub}(X)$ has finite joins and that these joins are stable under pullback (so substitution respects disjunction). A standard categorical package ensuring this is *coherence* (finite limits plus pullback-stable finite unions/images), but for our purposes it is enough to assume pullback-stable finite joins in the relevant fibers.

Syntax and rules. Under this additional hypothesis, for $A, B :: s^X$ we add

$$A \vee B : X,$$

with sort preservation:

$$\frac{\Gamma \,|\, \Delta \;\vdash\; A :: s^X \quad \Gamma \,|\, \Delta \;\vdash\; B :: s^X}{\Gamma \,|\, \Delta \;\vdash\; A \vee B :: s^X}\ \vee\text{-FORM}\ .$$

Membership rules are the usual natural-deduction rules:

$$\frac{\Gamma \,|\, \Delta \;\vdash\; t :: A}{\Gamma \,|\, \Delta \;\vdash\; t :: (A \vee B)}\ \vee\text{-I}_1 \qquad \frac{\Gamma \,|\, \Delta \;\vdash\; t :: B}{\Gamma \,|\, \Delta \;\vdash\; t :: (A \vee B)}\ \vee\text{-I}_2\ .$$

For any proposition $\psi(x)$ which may depend on $x : X$,

$$\frac{\Gamma \,|\, \Delta \;\vdash\; t :: A \vee B \quad \Gamma, x : X \,|\, \Delta, x :: A \;\vdash\; \psi(x) \quad \Gamma, x : X \,|\, \Delta, x :: B \;\vdash\; \psi(x)}{\Gamma \,|\, \Delta \;\vdash\; \psi(t)}\ \vee\text{-E}\ .$$

Soundness (joins and distributivity). We interpret $[\![A \vee B]\!] = [\![A]\!] \vee [\![B]\!]$ as the join in $\mathrm{Sub}(X)$. When the fiber is a distributive lattice (as it is in coherent/Heyting settings), distributivity holds propositionally (up to $\simeq$).

7.2.3 Adding $\Rightarrow$: Heyting and Boolean negation

Extra structure needed. To interpret implication, each fiber $\mathrm{Sub}(X)$ must be a Heyting algebra; equivalently, for each subobject $A \hookrightarrow X$, the meet-with-A operation has a right adjoint. Categorically, this is the usual "Heyting" strengthening of the subobject fibration (e.g. satisfied in a topos)[9, 8].

Syntax and rules. Under this hypothesis, for $A, B :: s^X$ we add

$$A \Rightarrow B : X$$

with sort preservation:

$$\frac{\Gamma \,\big|\, \Delta \;\vdash\; A :: s^X \quad \Gamma \,\big|\, \Delta \;\vdash\; B :: s^X}{\Gamma \,\big|\, \Delta \;\vdash\; A \Rightarrow B :: s^X} \Rightarrow\text{-FORM} \quad .$$

Crucially, this $\Rightarrow$ is *not* a function type: it is Heyting implication on predicates at carrier X. Its intended reading is pointwise:

$$t :: (A \Rightarrow B) \quad \text{means} \quad (t :: A) \Rightarrow (t :: B)$$

in the ambient internal logic.

$$\frac{\Gamma \,\big|\, \Delta, t :: A \;\vdash\; t :: B}{\Gamma \,\big|\, \Delta \;\vdash\; t :: A \Rightarrow B} \Rightarrow\text{-I} \qquad \frac{\Gamma \,\big|\, \Delta \;\vdash\; t :: A \Rightarrow B \quad \Gamma \,\big|\, \Delta \;\vdash\; t :: A}{\Gamma \,\big|\, \Delta \;\vdash\; t :: B} \Rightarrow\text{-E} \quad .$$

Negation. Once $\Rightarrow$ and $\bot$ exist, we define (Heyting) negation as an abbreviation:

$$\neg^X A \;:=\; A \Rightarrow \bot^{s^X}.$$

This is *pseudocomplementation*. In general $\neg^X \neg^X A \simeq A$ need not hold: that law is characteristic of boolean (classical) subobject lattices.

Boolean negation (optional). If, furthermore, each $\mathrm{Sub}(X)$ is a *Boolean* algebra (equivalently: every subobject has a complement), then $\neg^X$ is complement and we obtain excluded middle and double negation elimination up to $\simeq$.

Soundness for the extra structure fragment

Continue with the semantic setup of Subsection 5.7. Fix X and s^X. Under the additional hypotheses described above, the fiber $\mathrm{Sub}(X)$ supports the corresponding operations, and we interpret:

$$[\![\bot^{s^X}]\!] = 0, \qquad [\![A \vee B]\!] = [\![A]\!] \vee [\![B]\!], \qquad [\![A \Rightarrow B]\!] = [\![A]\!] \Rightarrow [\![B]\!], \qquad [\![\neg^X A]\!] = [\![A]\!] \Rightarrow 0.$$

With these interpretations, the rules in Subsections 7.2.1–7.2.3 are sound.

Lemma 7.2 (Heyting laws for logical type formers (conditional))**.** *Assume that for each carrier X and sort s^X the fiber* $\mathrm{Sub}(X)$ *is a Heyting algebra. Then the collection of type formers $A :: s^X$ modulo $\simeq$ forms a Heyting algebra under*

$$(\vee,\ \wedge,\ \Rightarrow,\ \top^{s^X},\ \bot^{s^X}),$$

and for all $A, B, C :: s^X$ the usual Heyting identities hold up to provable equivalence.

Sketch. These are exactly the laws of the Heyting algebra $\mathrm{Sub}(X)$ in the model; soundness follows from interpreting each former as the corresponding Heyting operation. □

8 Conclusion

We have described an endofunctor $\mathcal{H}_\Sigma$ on lambda theories that, starting from purely operational data (base rewrites and their contextual closure), systematically generates a hypercube of typed calculi. The modal layer associates to each base rewrite and each redex position a primitive modality $\langle K \rangle_{\vec{x}::\vec{A}} B$, with a rely-possibly reading that internalizes one-step behavior in a given context. Because rewrite steps do not support definitional conversion, we freely adjoin an empty-context possibility operator $\Diamond$ whose rules provide a weakened, rewrite-stable form of conversion: redex-in-context terms are typed at $\Diamond B$ when they can step to a term of type B. Equation-generated modalities, by contrast, admit full conversion, and in any cartesian closed setting the canonical evaluation equation yields a true dependent product Π.

Beyond the modal core, we have shown how to enrich the construction with structural and propositional layers without disturbing the underlying slot-and-center geometry. Structural type formers $f^\sharp$ classify terms by AST root and are interpreted as image subobjects of the corresponding formers, making it possible to speak cleanly about the spatial shape of processes inside the same internal language that speaks about their behavior. The propositional layer is stratified by the categorical strength of the base theory: in any finite-limits setting we obtain meet and top on type formers; with additional structure we can add bottom and use modal negation to express rely-inconsistency; with finite joins we recover distributivity; and with Heyting implication (and, optionally, Boolean structure) we recover full fiberwise negation and the familiar wealth of logical laws. Together, these layers carve out a spectrum that ranges from a small, Cube-like modal hypercube with a simple normalization theory, through increasingly rich fragments that approximate the full internal logic of a lambda theory, to the fully fledged Heyting (or Boolean) semantics available in stronger categorical settings[15]. This suggests a modular route to designing typed operational calculi: choose a lambda-theoretic presentation of the dynamics, choose how much propositional structure you are willing to support, and let $\mathcal{H}_\Sigma$ fill in the corresponding vertex of the hypercube.

References

[1] A. Abel and T. Coquand. Untyped algorithmic equality for Martin–Löf's logical framework with surjective pairs. In *Typed Lambda Calculi and Applications*, 2005. `https://www2.tcs.ifi.lmu.de/~abel/lfsigma.pdf`.

[2] J. Adámek, J. Rošický, and E. M. Vitale. *Algebraic Theories: A Categorical Introduction to General Algebra*. Cambridge Tracts in Mathematics. Cambridge University Press, 2010. `https://perso.uclouvain.be/enrico.vitale/gab_CUP2.pdf`.

[3] T. Altenkirch, N. A. Danielsson, A. Löh, and N. Oury. ΠΣ: Dependent types without the sugar. In *Functional and Logic Programming (FLOPS 2010)*, 2010. `https://www.cse.chalmers.se/~nad/publications/altenkirch-et-al-flops2010.html`.

[4] H. Barendregt. Introduction to generalized type systems. *Journal of Functional Programming*, 1(2):125–154, 1991.

[5] M. Barr and C. Wells. *Toposes, Triples and Theories*. Grundlehren der mathematischen Wissenschaften. Springer, 2013. `https://cjhb.site/Files.php/Books/%28Uncategorized%29/Before%202022/Toposes,%20Triples%20and%20Theories%20-%20M.%20Barr,%20W.%20Wells.pdf`.

[6] G. Boudol. The pi-calculus in direct style. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '97)*, pp. 228–242, 1997. `https://doi.org/10.1145/263699.263726`.

[7] P. Di Gianantonio, F. Honsell, and M. Lenisa. RPO, second-order contexts, and lambda-calculus. *Logical Methods in Computer Science*, 5(3), 2009. https://arxiv.org/abs/0906.2727.

[8] B. Jacobs. *Categorical Logic and Type Theory*. Studies in Logic and the Foundations of Mathematics 141. North Holland, 1999. https://annas-archive.org/search?q=jacobs+categorical+logic.

[9] P. T. Johnstone. *Sketches of an Elephant: A Topos Theory Compendium*. Oxford Logic Guides. Oxford University Press, 2002. https://cds.cern.ch/record/592033.

[10] J. Lambek and P. J. Scott. *Introduction to Higher-Order Categorical Logic*. Cambridge Studies in Advanced Mathematics. Cambridge University Press, 1988. https://books.google.com/books?id=6PY_emBeGjUC.

[11] J. J. Leifer and R. Milner. Deriving bisimulation congruences for reactive systems. In C. Palamidessi (ed.), *CONCUR 2000 — Concurrency Theory*, Lecture Notes in Computer Science, pp. 243–258. Springer, 2000. https://www.pure.ed.ac.uk/ws/portalfiles/portal/17894421/Leifer_and_Milner_2000_Deriving_Bisimulation_Congruences.pdf.

[12] S. Lybech. Encodability and separation for a reflective higher-order calculus. *Electronic Proceedings in Theoretical Computer Science*, 368:95–112, 2022. http://dx.doi.org/10.4204/EPTCS.368.6.

[13] L. G. Meredith and M. Radestock. A reflective higher-order calculus. *Electronic Notes in Theoretical Computer Science*, 141(5):49–67, 2005. https://www.sciencedirect.com/science/article/pii/S1571066105051893.

[14] R. Milner. The polyadic pi-calculus: a tutorial. 1993. https://api.semanticscholar.org/CorpusID:11201988.

[15] C. Williams and M. Stay. Native type theory. *Electronic Proceedings in Theoretical Computer Science*, 372:116–132, 2022. http://dx.doi.org/10.4204/EPTCS.372.9.